



12

Developing Others

Ian joined Toshi's team as a senior-level engineer with several years of experience gained at traditional enterprise companies. He impressed everyone with his technical skills. But during their first production incident, Toshi noticed that Ian froze when faced with live debugging.

Toshi recognized an opportunity to help Ian develop critical troubleshooting skills. The next day, when Toshi's on-call shift started, he casually invited Ian to sit with him. "I could use a second pair of eyes on these investigations," he said, framing it as him needing help rather than Ian needing training. He showed Ian how to form hypotheses quickly, then use targeted queries to test them and eliminate dead ends efficiently. He taught him how to balance urgency and quick mitigation with the thoroughness that digging deeper requires.

Over the next few weeks, Toshi made Ian the secondary on-call, and he paired with him when incidents occurred. He'd have Ian lead the investigation, and he would prompt him by asking guiding questions (e.g., "What would confirm this theory?"; "Where else might this symptom appear?"). When Ian got stuck, Toshi shared his mental models for different failure patterns, thus helping Ian to recognize common issues more swiftly.

diagnose issues faster. Toshi tracked Ian's progress through the changes he observed. Ian no longer had to escalate every complex issue because he could now resolve most of them independently. He also began to field questions from his teammates when incidents occurred. The team's mean time from incident to recovery was halved in that quarter, partly because Ian could now handle the calculation failures that previously only Toshi had understood. Toshi had helped Ian transform his existing strengths into practical troubleshooting skills, and that improvement had led to the entire team becoming more resilient.

What Does It Mean to Develop Others?

 **Developing others** means helping people build lasting skills rather than just solving their immediate problems for them. It focuses on building independent, capable teams.

Developing others means helping people build their problem-solving abilities and, consequently, their confidence over time. This is done by senior engineers who review code in ways that help their teammates understand design principles that they can apply to future projects. Architects who create learning opportunities to help others develop judgment about system trade-offs.

Team leads who structure projects to stretch people's capabilities and build new skills rather than always only completing tasks efficiently.

Microsoft builds "contributing to the success of others" into its core expectations. Your own performance isn't enough if you're not helping your teammates grow. Google embeds peer development into its engineering culture through readability reviews, design doc feedback, and a norm that senior engineers teach through code review; they don't just approve it. Meta wants engineers who build team skills alongside individual excellence. These companies understand that people who help others grow create more long-term value than people who focus only on their own output.

Developing Others



Teach principles not
just fixes



Build long term
capabilities



Team growth over
individual output



The Essence of Developing Others

thinking behind the answer they give. That pause, that choice to invest in someone else's growth, multiplies significantly the expert's impact.

The core challenge with developing others is balancing immediate needs with the benefit of long-term growth in others. An approach focusing purely on efficiency means doing tasks yourself or giving quick answers. At the other end of the spectrum, treating everything as a teaching opportunity can slow teams down when deadlines loom. Effectively developing others happens somewhere between those extremes. There is a time for quick guidance and a time to teach thoroughly; a time to step in and a time to let teammates wrestle with a problem; and a time to push people beyond their comfort zones.

Developing others also means recognizing that people learn differently and need different kinds of support. Some people thrive when granted independence; others need structured help. Some learn through documentation; others need hands-on pairing. Some people are cautious when accepting help from other people, especially peers or those with fewer years of experience; others will take any help they can get, regardless of level. A person who excels at developing others adapts their approach based on the needs, current skill levels, and learning styles of the individuals being developed. This often starts with asking how someone prefers to learn, but it also means staying flexible when that approach isn't working.

To be effective at developing others, one must be comfortable and patient with people's different rates of learning, and it is important to celebrate incremental progress. People who excel at developing others recognize small wins. They know that growth isn't always linear, and they will persist through periods when their investment doesn't seem to be paying off. They focus on trajectory rather than absolute performance because they understand that helping someone grow from struggling to being competent will create more value than the far easier job of polishing someone who is already strong. This patience has limits, though. When someone shows no movement at all despite varied approaches and genuine effort on both sides, the issue may not be the pace of learning but, rather, the fit between people.

Attempting to develop others can sometimes backfire. For example, your feedback may trigger a defensive roadblock, or a person's confidence might crumble during a stretch assignment. When your approach isn't working, vary your methods. If progress still stalls after genuine effort from both sides, the problem may be fit rather than ability. Sometimes you are not the right person to help someone, and recognizing that fact isn't failure. The section "When Developing Others Gets Difficult" covers these challenges in depth.

Cultural and Organizational Considerations

delegate it solely to managers. Understanding these differences will help you demonstrate this competency appropriately for your target company.

High-Growth Companies: At startups and scale-ups, speed is everything. There simply isn't time to wait for people to catch up on their own. These companies want to see that you can up-level the people around you without slowing down. The best stories show you teaching through the work itself: onboarding a new hire by pairing on a real feature rather than pointing them at docs, or helping a junior engineer with a production issue while shipping the fix on time. Success means the team gets faster *because* you invested in people, not *despite* it.

Large Tech Companies: These companies value more systematic talent development that scales across organizations. They want people who can both mentor individuals and create resources and frameworks that aid in the development of many people simultaneously. Talent development here includes building onboarding programs, creating technical workshops, and establishing learning cultures. Success requires balancing individual mentoring with scalable systems.

Consulting and Professional Services: Developing others directly impacts business success. Therefore, these organizations need people who can quickly upskill team members for new client engagements. Development happens through apprenticeship models where junior members learn by doing. Success means building skills that will increase the team's ability to take on more complex engagements.

Traditional Organizations: Development in a traditional organization often follows formal structures and defined career paths. These companies value people who work within established mentoring programs while also identifying additional growth opportunities.

Cultural Background Considerations: Companies differ in how they expect talent development to happen. Some value formal mentorship structures with clear senior-to-junior teaching relationships. Others expect peer-to-peer growth through code review, pairing, and shared ownership. Before your interview, learn which model the company favors. Job postings, engineering blogs, and Glassdoor reviews often signal this. Frame your stories to match. For example, if the company values collaborative development, emphasize how you helped peers grow through shared work. If they value structured mentoring, highlight how you created clear learning paths and tracked progress. The goal is to show that your approach to growing others fits how the company already operates.

Related Competencies

Developing Others vs. Learning: Your learning becomes more valuable when you can pass it on to others. A person with a lot of knowledge but an inability to teach effectively becomes a knowledge silo. The best professionals constantly learn to stay up to date while also helping others grow alongside them.

about their success, and then you can invest in their growth. Trust enables the admission of vulnerability that is needed for growth and development to occur. People need to feel safe admitting what they don't know and making mistakes while learning.

Mentoring People You Don't Manage

What if most of your talent development is with people you don't manage? For example, you help peers improve their skills, you teach cross-functional partners about technical constraints, or you guide teammates who report to someone else. This sort of talent development requires a different approach from that used when growing direct reports, since you can't formally assign learning or mandate improvement.

Growing others without authority becomes increasingly important at senior levels, where you are expected to help others through influence rather than control. Knowing how to help people grow when you have no formal power over them will distinguish you as a strong individual contributor who can up-level people without authority.

Five specific strategies will help in these scenarios: asking permission before offering guidance, establishing credibility through usefulness, translating concepts for different audiences, building mutual learning relationships, and creating scalable resources.

Ask Permission First

Many peers resist unsolicited advice. If a colleague mentions they're having trouble with their database migrations, your instinct might be to walk them through everything you know about schema versioning. But unsolicited teaching, however well-intentioned, usually gets a polite nod and no behavior change. You can actually damage a relationship by assuming someone wants your input.

The permission-first approach changes this. Before offering guidance, say, "I noticed you're wrestling with that migration. Want me to share what burned me last time I tried something similar?" Or "Would it be useful to think through that together?" Or simply, "Would you like a second opinion on that?"

This simple step respects their agency and can disarm any defensiveness they may have. When you give people a choice to accept your help, they're more open to learning from it. The framing matters, too. For example, whereas "What worked for me" sounds like sharing experience, "You should do it this way" sounds like criticism.

Watch for situations where someone hasn't asked for help but they clearly need it. You still need their permission, but you can initiate the conversation and make it easier for them to accept it. For example, "I noticed X happening. Would it be useful to have someone to talk through some approaches?" They can decline your offer, and that's fine, because forcing yourself on an unwilling recipient rarely works.

shipping ranking changes that degrade other results. But you barely know them because you've only worked together in standups. Starting with "Let me teach you about information retrieval" will come across as condescending. They won't trust your expertise if they don't know you.

Help them solve a real problem first. If they're stuck on a ranking bug that's affecting their team's metrics, offer to dig into it with them. Now you've shown your value through the work. Later, when you ask if they'd like to go through how to evaluate ranking changes more methodically, they're far more likely to say yes.

This approach builds teaching into the actual work. As you pair on their problem, you explain your thinking: "I'm checking recall and precision separately because overall accuracy can hide problems" or "Here's why I'd test this against long-tail queries first." You're solving their immediate problem while transferring knowledge. The teaching happens naturally because you're already collaborating.

Product managers, designers, and other cross-functional partners respond well to this approach. They rarely want technical lectures. But if you teach them relevant concepts while helping them ship their features, they're eager to learn.

Translate Technical Concepts for Different Audiences

Your product counterpart asks about why the recommendation engine is slow. Without thinking, you start explaining cache eviction policies, index selectivity, and query plan analysis. Their eyes glaze over within 30 seconds. You've lost them with implementation details when all they needed was, "The system gets slower as users add more items to their history, and we need two weeks to fix it."

Cross-functional partners need different explanations from what you would give engineers. They don't need to understand how something works; all they need to understand is why that "something" matters for their decisions. In the example above, the PM needed to hear "The system gets slower as users add more items, and we need two weeks to fix it," not "Here's how cache eviction and index selectivity interact with our query planner."

Use their domain language. For a designer asking about rate limiting, don't explain token bucket algorithms. They need to know that "users can only perform this action 10 times per minute, so your design needs an error state for when they hit the limit." If a sales engineer asks about latency, translate milliseconds into user experience: "Customers on mobile will notice a half-second delay on that screen." Give people just enough technical context to improve their decisions.

Focus on the practical implications. What decisions can they make more effectively with the knowledge you give them? What constraints will they need to design around? What trade-offs will affect their work? Give them just enough technical understanding to improve their judgment in their domain.

decisions will affect system quality.

Build Mutual Learning Relationships

One-way teaching can often feel condescending to peers because it positions you as "the expert" teaching them, which corresponds to them being less knowledgeable. This can lead to resistance even if you *are* the expert in the matter and your guidance is solid.

Frame mentoring as collaborative learning. Instead of "Let me teach you about system design," try "Want to work through this design together? I'm curious how you're thinking about state management." You're pitching it as both of you exploring the problem together. Humility helps enormously here. Share your own mistakes openly: "I got this wrong on my last project because..." or "I'm still figuring out the best approach for..."

Ask them for their expertise in something they are strong at. For example, the mid-level engineer you're helping with architecture might know the product domain far better than you. You could take the opportunity to learn from them about user workflows while teaching them about technical patterns. Similarly, the designer you're helping to understand system constraints could teach you about user research and interaction patterns.

Reciprocal teaching/learning relationships such as these will preserve peer status while allowing both people to grow. Neither of you is "the mentor" or "the mentee"; you're colleagues helping each other improve. This approach works especially well in flat organizations with many people at a similar level to you.

Scale Through Resources, Not Just Time

You can't help everyone who could benefit from your expertise. One-to-one support doesn't scale. If you help five engineers improve their observability practices through pairing and code review, that's great for those five people. But the thirty other people across adjacent teams could have used the same guidance.

You *could* try to mentor thirty more people, but it would be more effective and more practical if you were to create resources based on your work with those five engineers that would help others to learn without requiring your time. For example, you could write documentation on observability patterns and provide examples from your codebase. You could record a tech talk or make a video and share it widely. You could build guides that answer the common questions that the engineers asked you, so that more people can learn independently. Or you could create code reference examples that demonstrate good patterns.

individual engineers.

And then, once you've created the resource, keep it up to date. Add in answers to questions you hear repeatedly. If three people ask about the same topic, document it.

Build communities that encourage people to learn from each other. Start a weekly discussion in which engineers can share their war stories. Create a Slack channel for architecture questions. Run code review sessions that enable the team to learn together. Implement these sorts of things, and you will facilitate learning without being the bottleneck.

At senior and staff levels, this scalable approach is required. You can't have sufficient impact through one-on-one mentoring alone. You need to build systems and resources that will help many people grow simultaneously.

Building Growth Relationships Across Distance

Remote and distributed work removes the casual interactions and connections that happen naturally with proximity. For example, you can't observe someone struggling at their desk and offer them help. People don't overhear sessions between other teammates and learn from them. Therefore, growing others from a distance requires intentional structure.

Rather than simply trying to teach, start by building connections. Schedule regular one-on-ones focused on the remote workers themselves, not just the status of their projects. Learn how they prefer to communicate and when they're most available. Trust develops more slowly in the absence of in-person interaction, so you will need to invest more time upfront.

Create async learning opportunities that can work across time zones. Record explanations of complex topics that they can review on their schedules. Write detailed code review comments that start conversations where you can teach principles and not just point out issues. You can write shared documentation that captures your thinking processes. These sorts of artifacts help when you can't be available in real time.

Watch for signals that indicate someone needs more support. Remote team members may tend to struggle through problems without admitting it because they don't want to make it seem like they can't handle remote work. Check in on how they're doing, not just what they're delivering.

The Core Challenge

The common thread across all five of these strategies is that you have to earn the right to help someone grow. You earn it by being useful first, by asking rather than assuming, and by making the learning mutual. You can't require anyone to learn from you when you don't manage them. You can only create conditions where they want to. At senior and staff levels, this skill becomes essential

functional partners, and people across teams through influence and shared resources will multiply their impact across the organization.

When Developing Others Gets Difficult

Even experienced people will encounter situations where their well-intentioned help doesn't go as planned, because growing others rarely follows a straight path. Understanding the common challenges will help you recognize problems early, so that you can adjust your approach. Interviewers will view favorably candidates who can describe both their successes and the times they had to recover when things went awry.

Common Mentoring Failure Modes



Resistance Wall
Defensiveness



Dependency Trap
Keeps coming back



Over-
Investment Spiral
Time sink



Style Mismatch
Not clicking



Misaligned Mentoring
Wrong skill

The Resistance Wall

Your colleague becomes defensive when you try to coach them. They shut down, they make excuses, and they dismiss any feedback you offer them. The relationship begins to fray because you try to initiate conversations to help them build new skills. Then the person you're trying to help starts avoiding you.

These sorts of responses happen when you haven't "earned the right" to offer to mentor a person, when your timing is off, or when your delivery feels like criticism to them rather than support. Sometimes the problem is that they don't yet trust you. Other times, it might be their ego or fear of looking incompetent. Cultural differences in how people give and receive feedback can also create resistance.

helpfulness: answer a question in Slack, share a useful article, offer to pair on something they're stuck on. You're re-earning trust, not re-establishing a teaching relationship. Only after they're comfortable working with you again should you cautiously revisit any kind of coaching dynamic; and even then, ask first.

In interviews, showing that you know how to recover from setbacks to your mentoring attempts will show your emotional intelligence and adaptability. Strong candidates will show that they recognize when their approach isn't working and will adjust it rather than simply pushing harder.

The Dependency Trap

If the person you're helping keeps coming back for the same type of help, they're not developing independence; rather, they've developed a dependency on you. You answer their questions, they complete the task, then return with a nearly identical question the next day. This happens when you solve problems for people instead of teaching them how to approach problems. Maybe you give answers because it's faster. Sometimes you remove the productive struggle too early, preventing them from building their own problem-solving ability.

Shift from giving specific answers to teaching general frameworks. When they ask how to investigate an issue, respond with "What's your hypothesis?" and "How would you test that?" Teach the approach, not the solution. Gradually increase the space between check-ins so they have room to struggle productively. Celebrate their independent attempts, even when they're imperfect. Document the frameworks you teach so they can reference them without coming back to you. The goal, and what interviewers want to hear, is that you understand that independence is the measure of success, not how often people seek your help.

The Over-Investment Spiral

You realize that you've been spending a disproportionate amount of time helping one person while the rest of your team has languished. The time investment has grown gradually, making it hard to notice until the imbalance has become problematic. Team members notice you're always busy with this person and wonder if their growth matters less to you. Other priorities have slipped because you've been too invested in helping this one individual. Even your own deliverables have begun to fall behind.

The sunk cost fallacy drives this damaging pattern. You've already invested significant time, so you reason that investing more will finally make it pay off. It becomes personal, and your judgment about the success of your efforts is clouded. You avoid the hard conversation about whether this person can succeed in their role.

you're not the single point of dependency. Track the return on time spent compared to other ways you could build skills. Know when to escalate to management for performance conversations that may be needed. Remember that helping just one person well helps one person, but creating scalable talent development systems helps many people.

Structuring your support in this way shows judgment about resource allocation. Strong candidates will describe how they balance the needs of individuals with broader team and organizational needs.

The Style Mismatch

Your natural teaching method isn't working. You're putting in the effort, and your teammate is trying to learn, but they're not improving. Your go-to methods that have worked for others aren't clicking for this individual.

Different people have different learning styles. Some are visual learners; others learn through conversation. Some want theory first, and others need hands-on practice before concepts make sense to them. Cultural differences in communication affect how people process information. If you're not sensitive to the differences of the people you're teaching, you may find yourself teaching them in a manner that does not suit them.

Ask the people you are helping directly how they learn best. If your initial attempt does not seem to work, try a different approach. Find others who might connect better with this person. Be honest if you think you might not be the best fit for them. Sometimes the right move is to connect them with someone else.

Interviewers like to see evidence of your self-awareness and creativity and that you can adapt your methods to suit individuals rather than expecting everyone to learn your way.

Recognizing Problems Early

Watch for these warning signs that your talent development isn't working: repeated questions about the same topics, decreasing engagement in conversations, lack of observable changes in behavior, increasing time investment without progress, or the person seeming to humor you rather than genuinely learning.

When you notice any of these signs, pause and diagnose before investing more time. Ask the person if they think your involvement is helping. The best recovery may be admitting that your current approach isn't working, and regrouping.

The best stories include examples of both successes and thoughtful recoveries. Describing how you recognized a problem and adjusted your approach will show your maturity and adaptability, both qualities that interviewers value highly.

Interviewers will explore your ability to develop others by asking questions that probe how you help others build skills. They want to know if you help grow independent, capable colleagues or you just answer their questions when asked.

Supporting Underperformers

- "Tell me about a time you helped an underperforming team member improve."
- "Describe a situation where someone on your team wasn't meeting expectations. What did you do?"
- "Give me an example of turning around someone's performance."

These questions test your ability to have difficult conversations and support improvement when it matters most. Interviewers want to see diagnosis before action. Do you seek first to understand the root causes of underperformance, rather than assuming there is a problem with motivation? Strong answers will show you giving clear feedback delivered with empathy, implementing tailored improvement plans with measurable milestones, and being patient with the person's rate of improvement. Also, do you know when performance issues require management escalation, and do you appreciate that underperformance often signals missing skills, unclear expectations, or personal challenges rather than merely a lack of effort?

Helping People Grow

- "Tell me about a time you helped someone advance their career."
- "Describe how you've helped a team member build technical skills."
- "Give me an example of investing in someone's long-term growth."

These questions assess whether you understand the development needs of others and can create real growth opportunities. Interviewers want to see structured approaches to helping others advance, rather than ad-hoc mentoring. Strong answers will show you identifying specific growth areas in others, creating learning opportunities for them that will stretch their capabilities, and tracking their progress over time. This will show your willingness to invest in other people's success beyond immediate project needs.

Giving Helpful Feedback

- "Give me an example of providing feedback that helped someone improve."
- "Tell me about a coaching relationship you've had with a team member."
- "Describe a time when you had to deliver difficult feedback to help someone grow."

When interviewers ask questions like these, they seek to evaluate your ability to give guidance that changes behavior. Your answers to these questions will show whether you can deliver constructive feedback effectively and adapt your approach to how people learn. Good examples will show you

Cross-Functional Development

- "Tell me about a time you developed someone outside your direct technical domain."
- "Describe how you've helped non-engineers develop technical understanding."
- "Give me an example of mentoring someone in a different role than yours."

Strong answers to these questions will show your skill at translating complex details. For example, you can explain technical concepts without using jargon, you can employ relevant analogies, and you focus on practical application over textbook understanding. Good examples will show your patience with different learning styles and recognition that non-technical colleagues need different onboarding than engineers. You might describe helping a product manager understand why certain machine learning features need weeks of training data before they're useful, so they can set realistic launch timelines. Or teaching a designer that real-time notifications have a cost per user, so they can weigh notification frequency against infrastructure budget. Or showing an ops team how to read deployment dashboards so they can triage customer issues without escalating to engineering.

Remote and Distributed Development

- "How have you developed team members working remotely or across time zones?"
- "Tell me about adapting your mentoring approach for distributed teams."
- "Describe challenges you've faced developing people you don't see in person daily."

These questions assess whether you can build productive relationships without physical proximity. Strong answers will show intentional communication practices such as scheduled check-ins, written documentation, and async tools for knowledge-sharing. Good examples will show your creativity in providing teaching moments through recorded videos, async pair programming sessions, or written technical discussions. You might describe establishing a weekly technical blog on which remote team members shared their learnings, or creating a mentorship matchmaking tool to connect people across time zones. Interviewers want to see that you build relationships and trust before trying to develop people remotely, and that you understand the challenges that remote team members face relating to isolation, beyond just their technical work.

Building Stronger Teams

- "Tell me about building a team with complementary skills."
- "Describe how you've raised the technical bar on your team."
- "Give an example of creating systems that help everyone improve."

With these questions typically targeted at seniors and above, interviewers want to assess whether you think strategically about collective growth. They want to see if you build environments where people develop others through working together. The best stories will show you designing team structures

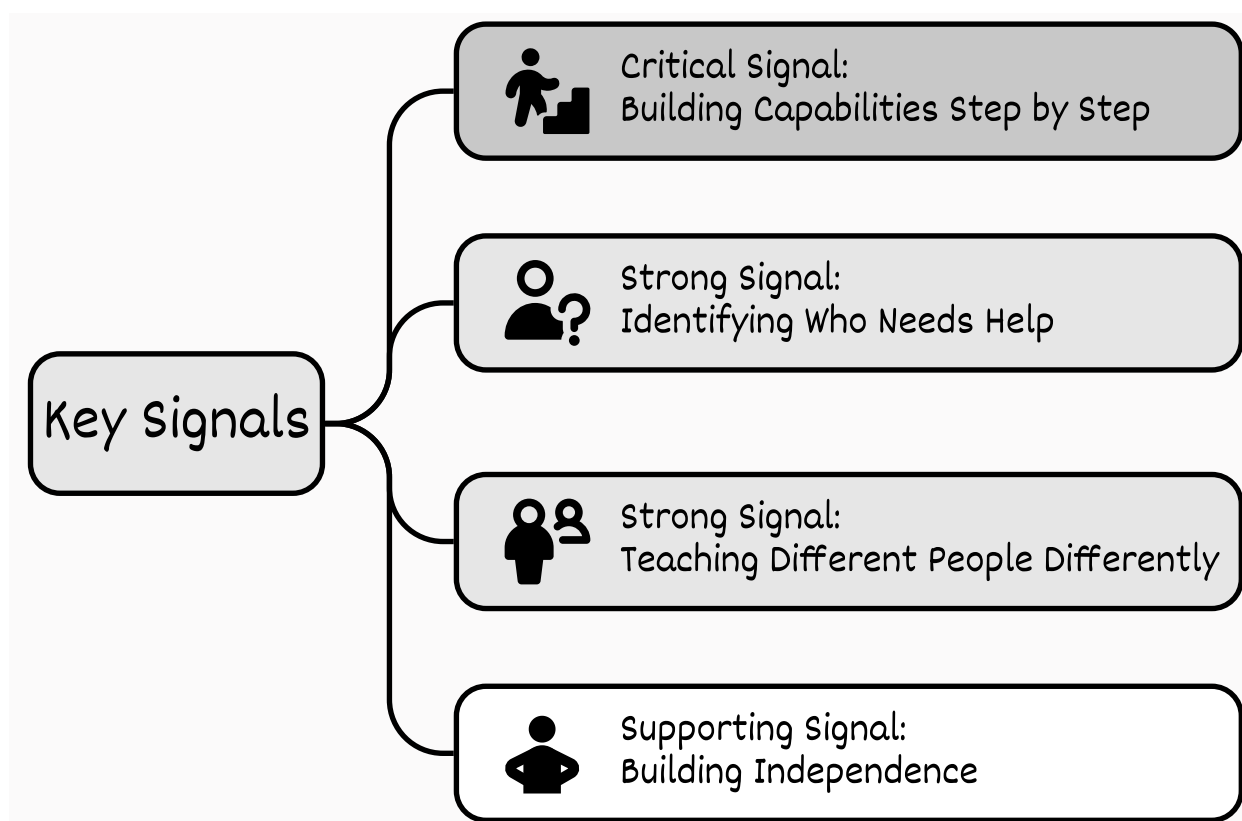
When Developing Others Gets Hard

- "Tell me about a time when developing someone didn't go as planned."
- "Describe helping someone who was struggling to improve."
- "Give an example of adapting your approach to developing someone when it wasn't working."

These questions are designed to assess your persistence and creativity when developing others gets difficult. Interviewers want to see you trying multiple approaches rather than giving up when your standard methods fail. Strong responses will show you diagnosing why an approach has stalled, trying different teaching methods, and sometimes accepting when someone might need support from someone other than you.

Key Signals

When evaluating your development skills, interviewers look for evidence that you build skills through structured approaches and that you don't just share knowledge occasionally. The strongest candidates will show that they create lasting growth in individuals and teams.



Critical Signal: Building Capabilities Step by Step

Effective development requires breaking complex skills into learnable pieces and creating experiences that build confidence alongside competence. Strong stories will show you identifying where a person is starting from, designing challenges appropriate to their level, and helping them progress through

Strong Signal: Identifying Who Needs Help

People who excel at developing others notice patterns in questions, gaps in approaches, and unrealized potential. Good examples will show you initiating conversations, identifying the need for growth before it becomes a limitation, and helping people see possibilities they hadn't considered for themselves.

Strong Signal: Teaching Different People Differently

Effective mentors and coaches adjust their methods based on each person's background, learning style, and motivations. Strong examples will show you recognizing when your default approach isn't working, trying different explanations when someone doesn't understand, and creating varied learning experiences to suit different personalities.

Supporting Signal: Building Independence

Success is achieved when people can solve new problems without you, not just repeat what you taught them. You should be able to show that you teach people how to think about problems rather than merely provide them solutions, and that you gradually reduce your support as they become more capable and confident.

Red Flags

We use Red Flags to describe serious issues that will significantly damage your candidacy and Yellow Flags for concerning patterns that will weaken your examples.

Many candidates confuse simply helping people with developing talent, thus undermining their stories. Understanding the following differences will help you focus your answers on examples of up-leveling others.

Red Flag: Giving Answers Instead of Building Skills

Solving problems for people is helpfulness, not talent development. If someone repeatedly comes to you with the same type of question, that is an indication that you're creating dependence. Success means people are able to handle subsequent similar challenges on their own. Focus on telling stories that describe your teaching approaches and frameworks, not just meting out solutions.

Yellow Flag: Development Without Business Context

Teaching skills that are unrelated to a person's actual work needs suggests that you prioritize learning for learning's sake over results. Talent development bandwidth is limited, so it should be applied to

Yellow Flag: Surface Teaching Without Depth

Running training sessions or sharing documentation without confirming that there was a subsequent growth in capability suggests incomplete development. Exposure to information isn't the same as developing skills. Include in your stories evidence that people improved. Were they now able to solve new problems? Did they take on new responsibilities? How did the quality of their work increase?

Yellow Flag: Taking Credit for Natural Growth

Claiming responsibility for growth that would have happened anyway weakens your credibility. People naturally improve with experience, so you need to demonstrate that your specific contribution went beyond what would have happened naturally. Therefore, focus your stories on breakthrough moments that you facilitated or the abilities that people developed as a consequence of your specific intervention.

Example Stories by Level

Developing others becomes a significant expectation for roles starting at the senior level. While junior and mid-level engineers might help their teammates informally, companies will typically look for evidence of talent development capabilities when evaluating for senior-plus roles. This competency is also especially important for promotion to staff and principal levels, where your impact increasingly comes via scaling through others rather than just your individual contribution.

Senior-Level Example: Building Technical Investigation Skills

Question: "Tell me about a time you helped someone develop their technical skills."

Headline: "Earlier this year, I helped a mid-level engineer get up to speed in record time for our complex financial calculation engine."

Key Point 1: "A new engineer joined our team working on high-frequency trading software. He had strong coding skills but struggled when bugs appeared in our concurrent calculation engine. I noticed he kept asking the same types of questions about race conditions and precision errors. He'd stare at thread dumps without knowing where to start. Rather than just answering each question each time, I saw he needed to learn how to debug concurrent systems, not just fix individual bugs."

Key Point 2: "I started by having him shadow me while I debugged a race condition in our order matching logic. I explained my thought process out loud as I used debuggers to examine thread states and trace execution paths. Next, we paired on simpler issues. I got him to do the debugging while I asked guiding questions. We progressed from single-threaded calculation errors to race conditions to

Key Point 3: "Instead of solving problems for him, I taught him patterns. When he hit issues, I'd ask 'What's your hypothesis?' and 'How would you test that?' I showed him how to use thread sanitizers, interpret stack traces, and recognize common concurrent programming bugs. By month three, he was debugging complex issues independently and had created an onboarding guide for future team members."

Landing: "He went from being overwhelmed to becoming our team's go-to person for tricky issues. Five months later, he diagnosed a subtle race condition that had been causing intermittent trading discrepancies for weeks."

Staff-Level Example: Data Engineering Capability-Building

Question: "Describe how you helped build capabilities across your organization."

Headline: "I helped our product teams develop data engineering skills when our company's data needs exploded but we couldn't double the size of that team."

Key Point 1: "As a staff data engineer, I saw that our product teams were waiting for weeks for data pipeline changes while our team was busy handling critical infrastructure migrations and real-time analytics for critical business metrics. We couldn't hire our way out of this problem. The data team needed to focus on this high-impact platform work, not routine, but still important, pipeline requests. Our product engineers had the technical skills to write code, but they lacked knowledge about modeling, pipeline reliability, and analytics patterns. Each team was making the same basic mistakes with schemas and data quality. I realized we needed to enable them to meet their own data needs."

Key Point 2: "I created the Data Engineering Toolkit, a resource with different learning paths tailored to the needs of different teams. Backend teams who understood distributed systems learned through building streaming pipelines using familiar patterns. Teams dealing with transactional data started with dimensional modeling before moving to pipeline concepts. For teams that learned best through problems, I created data quality challenges based on real incidents. Some engineers preferred theory first, so I ran sessions on slowly changing dimensions and data warehouse patterns. Thus, each team developed the same capabilities through methods that matched their existing systems."

Key Point 3: "The program built skills progressively over four months. Month one covered SQL optimization and window functions using their actual data. Month two introduced data modeling through redesigning their team's schemas with proper fact and dimension tables. Month three taught pipeline building, starting with simple batch jobs and progressing to handling late-arriving data. The final month covered data quality monitoring and debugging pipeline failures. Each stage included real work on their team's data problems, with peer reviews to share learnings across teams."

standard needs. This meant that the data team could focus on platform improvements and complex analytics that drove business decisions. The product teams now catch data quality issues during development rather than in production reports. The peer review network continues today, with teams sharing data modeling patterns and pipeline solutions. "

Principal-Level Example: Building Organizational Talent Development

Question: "Tell me about how you transformed how your organization develops technical talent."

Headline: "I convinced leadership to invest in technical leadership development and built a program that enabled us to scale our organization from 60 to 300 engineers."

Key Point 1: "As we scaled rapidly, I noticed a pattern developing. We had excellent individual contributors, but few could lead complex technical initiatives or develop others effectively. Senior engineers struggled with architectural decisions affecting multiple teams. They solved problems well in their domain, but they didn't know how to build consensus or teach strategic thinking. I documented the impact through failed cross-team projects, repeated architectural mistakes, and senior engineers leaving because they didn't see pathways for growth. I presented this data to our VP of Engineering, making the case that developing technical leadership was as critical as hiring."

Key Point 2: "With VP sponsorship and budget secured, I assembled partners to design the Technical Leadership Fellowship. I brought in our most effective principal engineers to define core technical leadership competencies. I partnered with an external leadership coach who specialized in engineering organizations. My contribution was analyzing our past technical successes and failures to identify the specific leadership capabilities we'd need for upcoming challenges, for example, scaling internationally and building new product lines. I ran the case study sessions myself, using our actual technical decisions as teaching materials. The six-month program selected twelve high-potential senior engineers as fellows. They presented technical decisions to senior audiences, led cross-team initiatives without formal authority, and developed others through complex projects."

Key Point 3: "We designed multiple learning approaches into the program. I ran architecture case study sessions for systems thinkers using our company's real technical decisions. The external coach provided one-on-one sessions for engineers struggling to develop influencing skills. We created peer learning cohorts where fellows would share their experiences and challenges. When fellows had specific gaps, we connected them with leaders who had previously overcome similar challenges. For example, when an infrastructure engineer struggled with product thinking, we paired them with a leader who'd made that transition in the past. The program adapted to the needs of individuals while maintaining consistent outcomes."

Landing: "The fellowship has developed 24 technical leaders across six engineering domains, with 18 taking on significant leadership roles. The VP who sponsored the program credits it with enabling our

improved retention of senior engineers. Fellowship alumni now run scaled-down versions of the program within their sub-organizations, thus multiplying the impact."

If Examples Don't Come Easily

You don't need "manager" in your title to have great talent development stories. Think about when you helped a struggling teammate finally understand a concept they'd been fighting with. Or when you created documentation that became the team's primary resource for onboarding. The patience you showed to explain something differently for the fifth time before it clicked might be exactly the example you need.

Have you ever taught someone a skill that made them independent in an area where they had previously needed help? That shows you can build lasting skills. Have you created learning opportunities that have stretched people beyond their comfort zones? That shows thoughtful development. Have you helped someone transform from struggling to a strong contributor? If you have invested in the growth of other people and seen them succeed, you have demonstrated strong talent development skills.

Reflection Questions:

- When have you helped someone master a skill they initially struggled with?
- What systems or resources have you created that have helped multiple people learn?
- Have you adapted your teaching approach for different learning styles?
- When did you identify opportunities for growth that others hadn't recognized in themselves?
- What development have you provided while managing your own deliverables?
- Have you helped people work through confidence issues or imposter syndrome?
- When have you turned your expertise into teachable frameworks?
- What lasting skills have you built in team members?
- Have you developed others informally without formal mentoring relationships?
- When did investing in someone's growth pay off in unexpected ways?

Key Takeaways

 **Developing others** means building lasting abilities rather than creating dependencies. Your goal is independence, not helpfulness. Effective talent development requires you to adapt your approaches to individual learning styles and needs and understand that what works for one person may confuse another. You must balance immediate efficiency with long-term team growth, discerning when to solve problems quickly and when to invest in teaching. At senior

Strong Developing Others Stories Include:

- Evidence of helping people become independent in areas in which they had previously struggled.
- Examples of adapting approaches for different people.
- Clear progression, showing how people grew over time.
- Creation of reusable learning resources or systems.
- Balance between individual mentoring and scaling through others.

Avoid These Traps:

- Don't describe merely giving answers without building skills.
- Don't use one-size-fits-all approaches to growing others.
- Don't focus only on star performers while ignoring struggling team members.
- Don't claim credit for natural growth that would have happened anyway.
- Don't develop others at the expense of your own deliverables.

☐ Mark as Complete[Teach on ByteByteGo](#)[Be an Affiliate](#)[Become a Contributor](#)hi@bytebytego.com[Report a Bug](#)[Our Team](#)[Newsletter](#)[Privacy Policy](#)[Terms of Service](#)



YouTube

Visual Dev Guides

Find Jobs

Prepare for Coding Interviews

Copyright ©2022-2026 ByteByteGo Inc.
All rights reserved.